

Expectations for Students

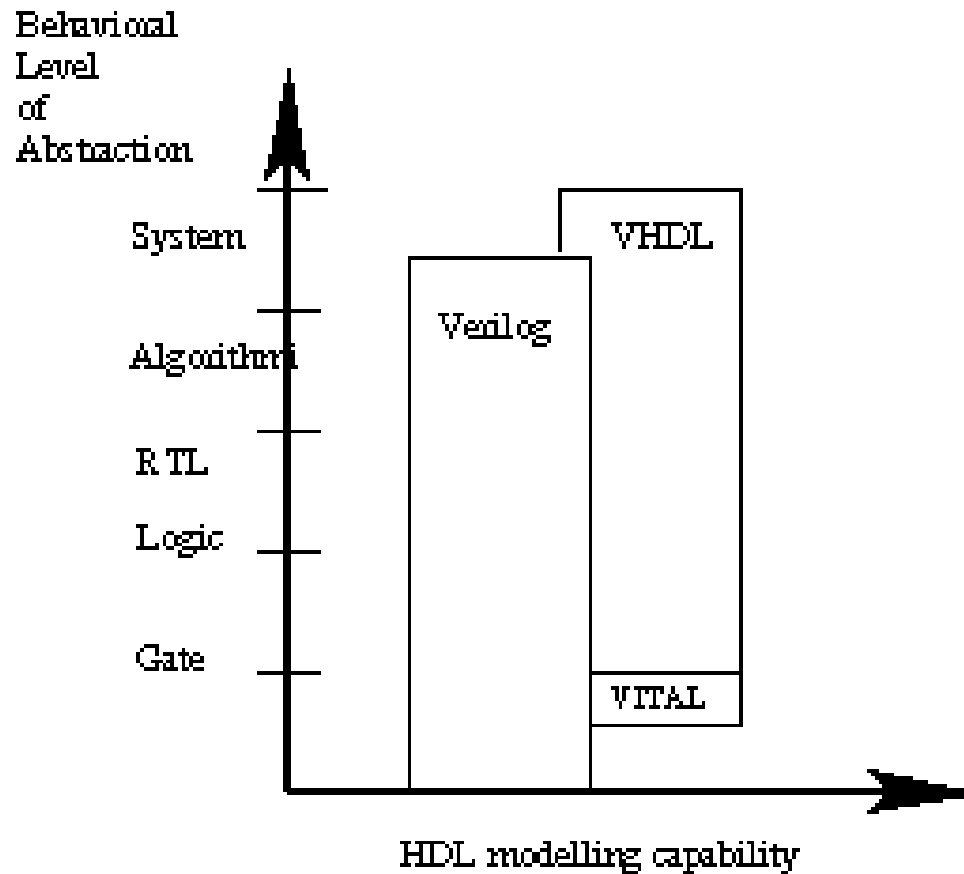
What you should know:

- Minimum: Digital logic circuits:
gates, flip-flops, registers, counters,
simple logic functions, etc.**
- Helpful: computer architecture, finite
state machines, logic minimization,
truth tables, etc**
- Helpful: Programming knowledge, esp.
C, etc**

What you should learn:

- **Logic design review**
- **Write Verilog models**
- **Concurrent statements**
- **Sequential statements**
- **Types and operators used in synthesis**
- **Common RTL (Register Transfer Level)
AKA synthesis style**
- **Simple methods for testbench construction**

VHDL vs. Verilog



- personal preferences
- EDA (electronic design automation) tool availability
- commercial, business and marketing issues

Logic Introduction

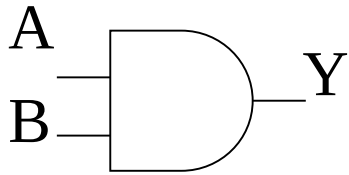
- **Logic circuits use two (i.e. binary) values: 0/1, low/high, or false/true**
- **Logic functions (AKA operators) give outputs based on the values of the inputs (common operators are AND, OR, NOT, NAND, NOR, XOR, XNOR)**
- **Logic element: gate – Hardware implementation of function - Output depends on the current values of the inputs**
- **Logic element: flip-flop (AKA register) or latch - Output depends on the sequence of inputs rather than only the current state.**

Simple Function Truth Tables

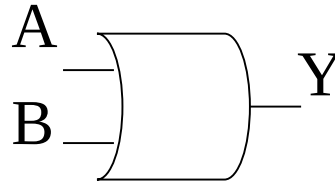
A	BUF	NOT
0	0	1
1	1	0

A	B	AND	OR	XOR	NAND	NOR	XNOR
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

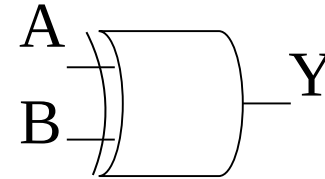
Schematic/Verilog Representations (combinatorial)



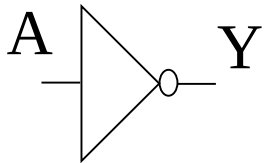
$$Y = A \& B;$$



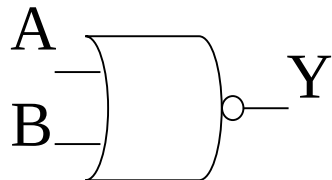
$$Y = A | B;$$



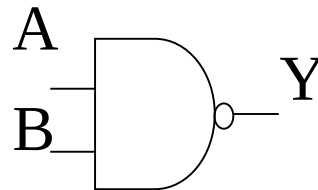
$$Y = A \wedge B;$$



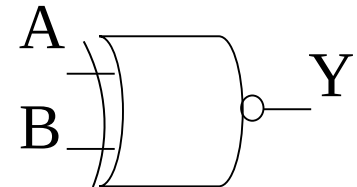
$$Y = \sim A;$$



$$Y = A \sim | B;$$



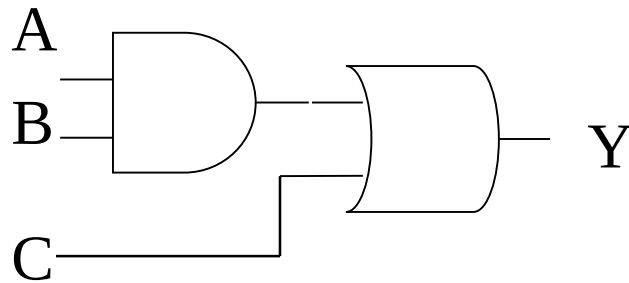
$$Y = A \sim \& B;$$



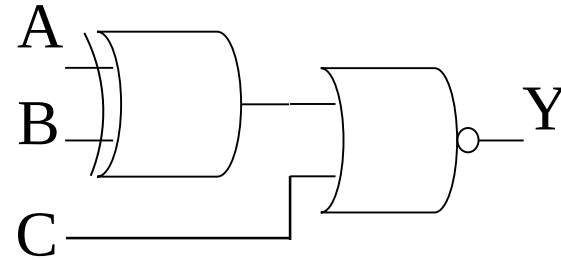
$$Y = A \sim \wedge B;$$

Vivado not support $\sim|$, $\sim\&$

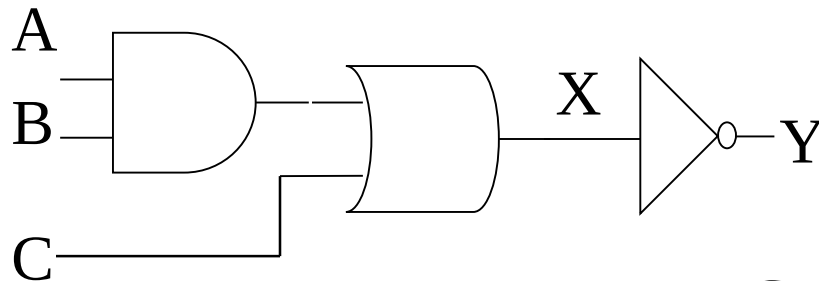
Schematic/Verilog Representations (combinatorial)



$$Y = (A \& B) | C;$$



$$Y = (A \wedge B) \sim | C;$$



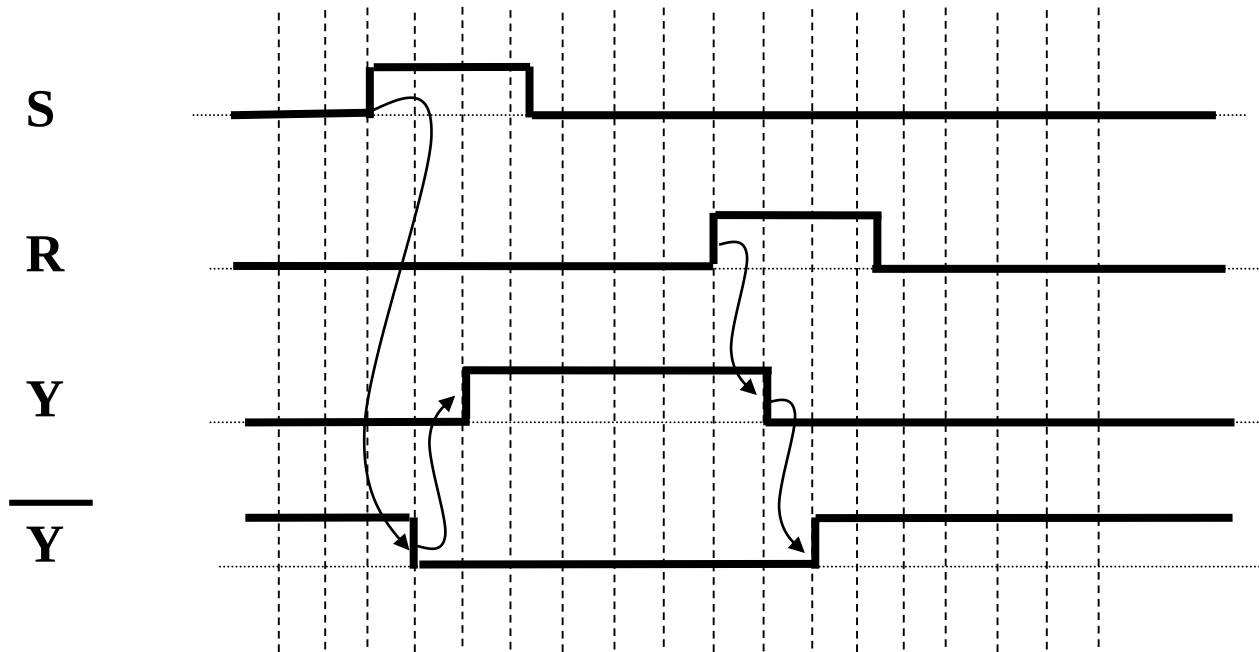
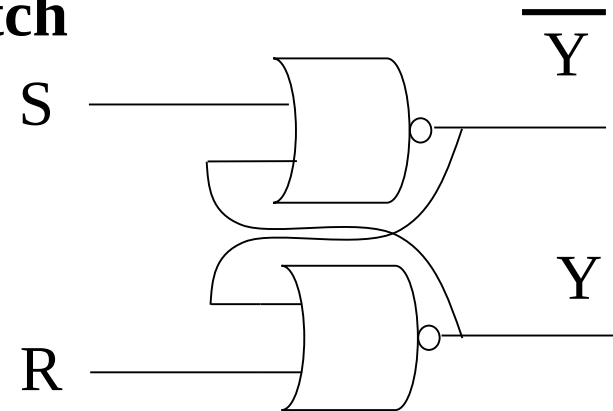
$$Y = \sim X;$$

$$X = (A \& B) | C;$$

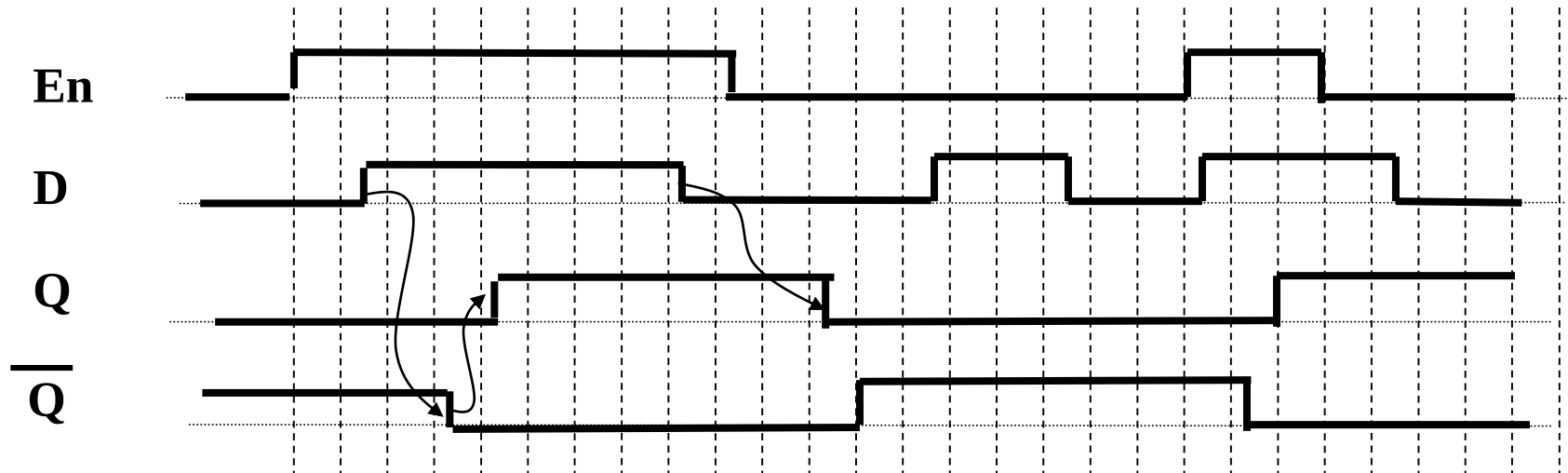
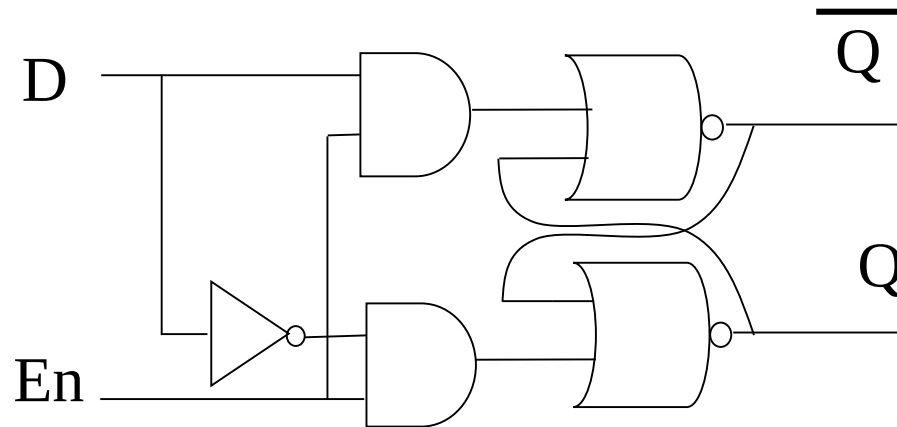
Code order does not matter

Storage with Latch

A latch (or flip-flops) stores one bit (binary digit)

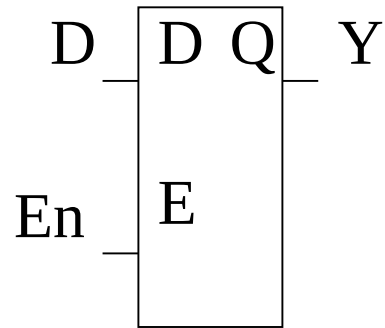


D Latch with Enable

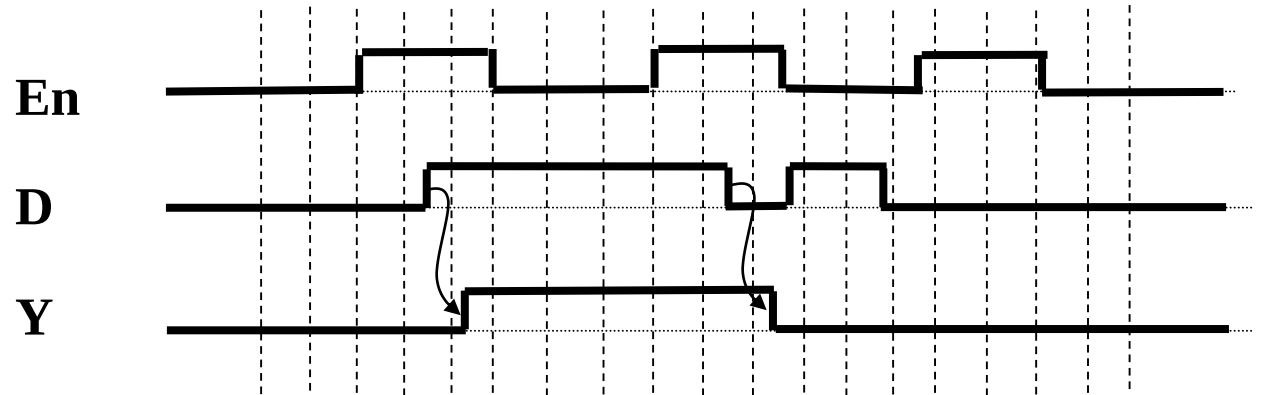


Latch vs. Flip-flop

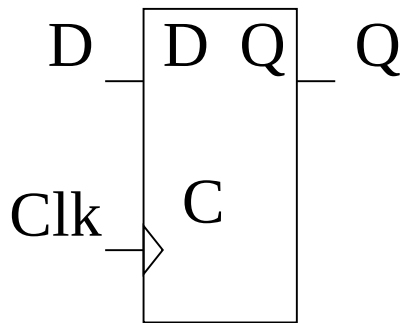
D Latch



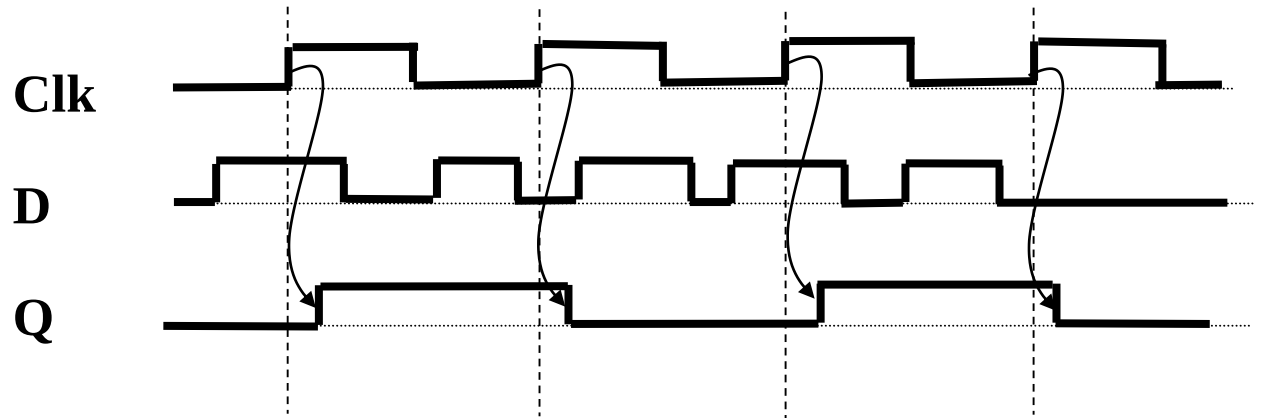
Level sensitive



D Flip-flop

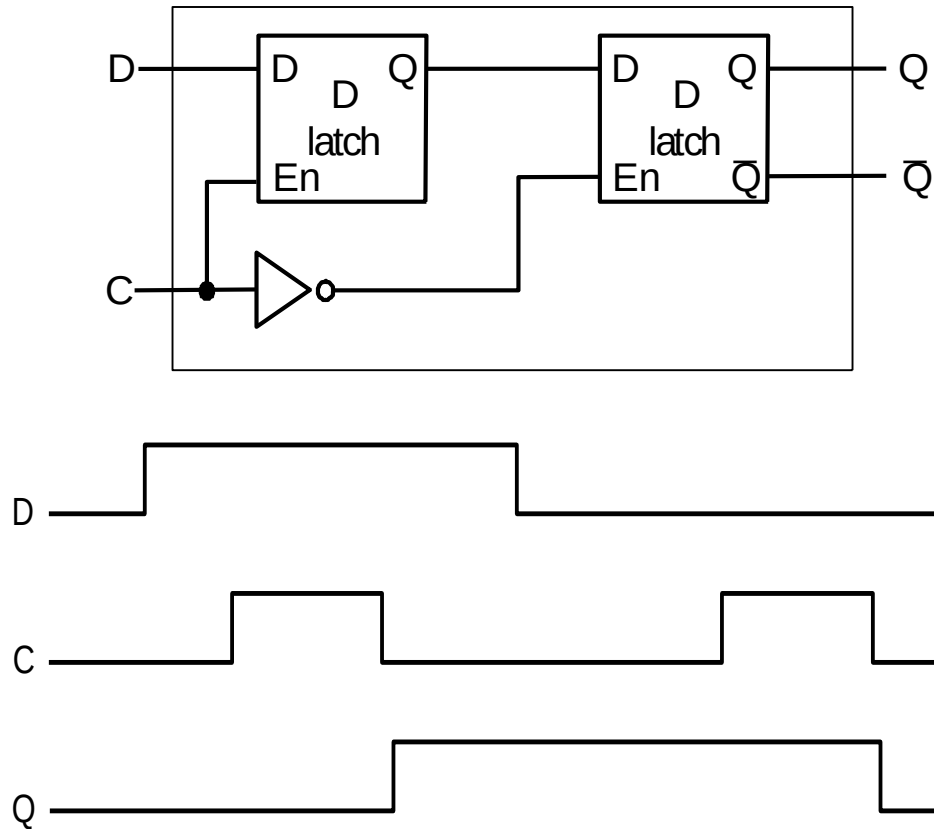


Edge-sensitive



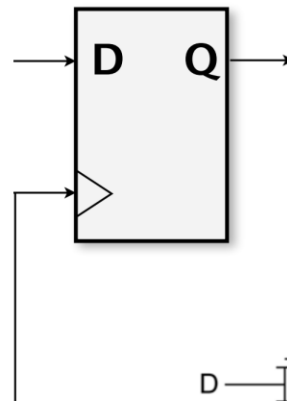
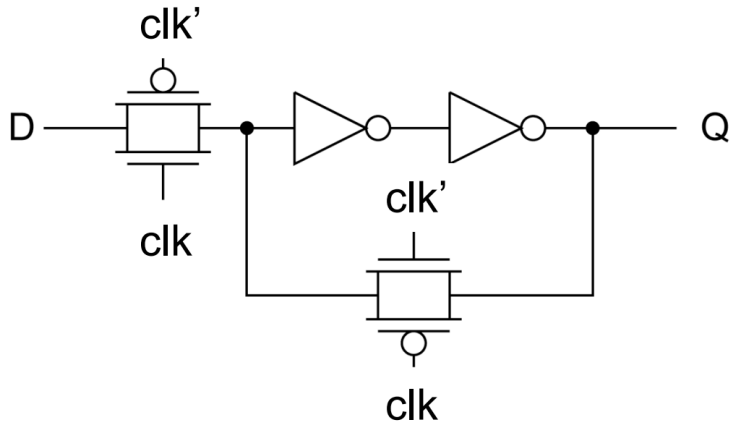
D flip-flop

- Output changes only on the clock edge



D FF Implementation

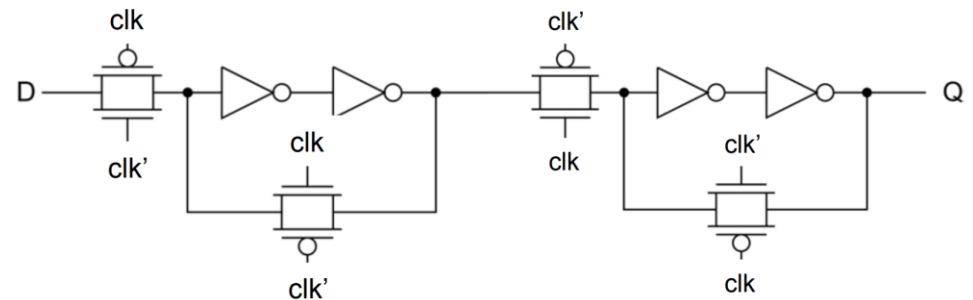
Latch Implementation:



A flip-flop “samples” right before the edge, and then “holds” value.

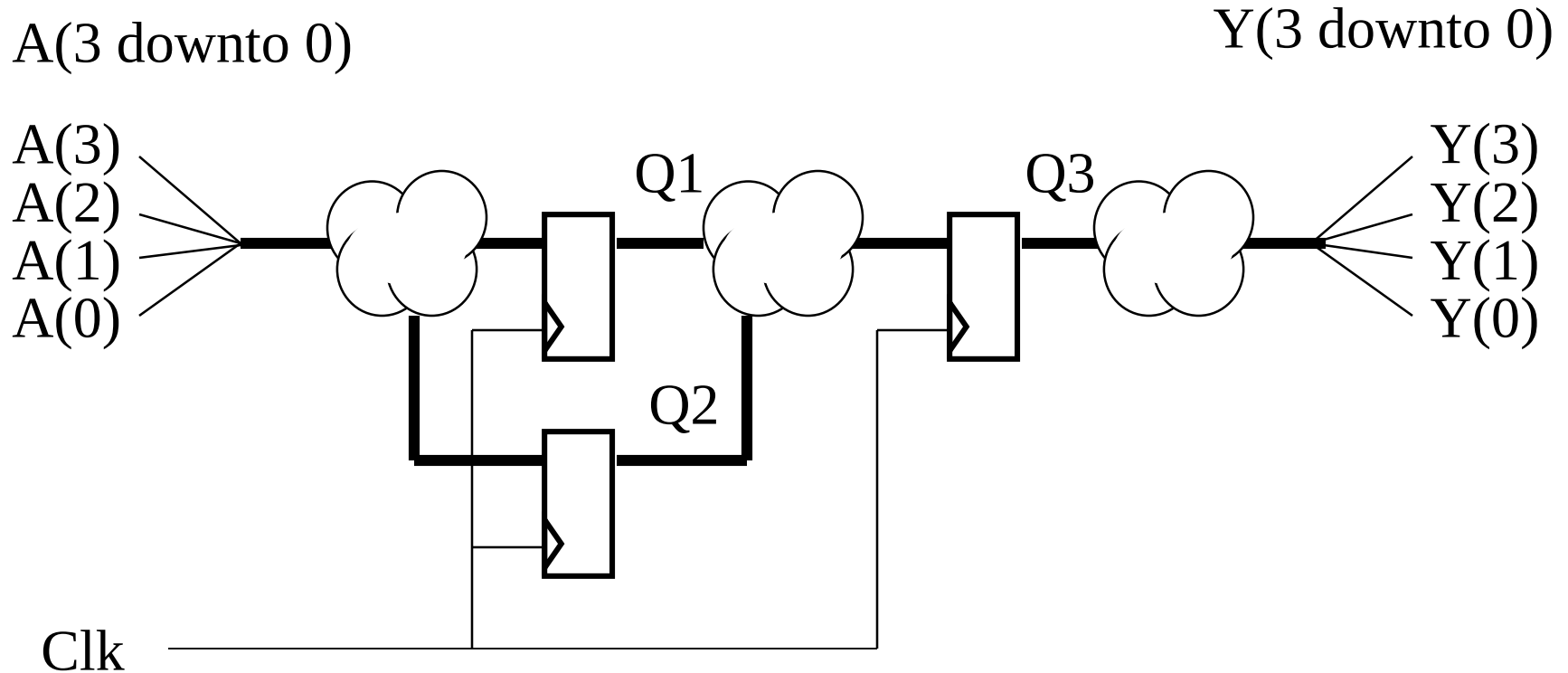
Sampling circuit

Holds value



Register Transfer Level (RTL)

Registers and Combinational Logic (Gates) between the registers



How Verilog is Used

ASIC: Application Specific Integrated Circuit (nonvolatile device)

FPGA: Field Programmable Gate Array

High speed (32.75 Gbps I/O),

High density (288 Million gates)

Three Major FPGA Vendors:

Xilinx: SRAM based FPGA, In System Programmable (ISP) device, need on board nonvolatile configuration memory to save bit stream file to configure FPGA when power is on.

Altera: SRAM based FPGA (High density PLD, ISP), now Intel

Actel: Antifuse based FPGA, One Time Programmable (OTP) device and Flash memory based FPGA, (ISP). nonvolatile device.

Design Automation Industry (2011)

Top companies

\$5.77 billion - [Synopsys](#)

\$4.46 billion - [Cadence](#)

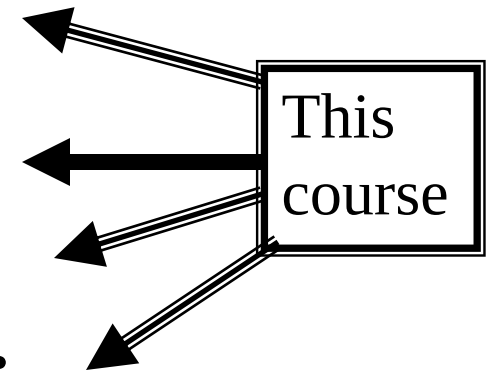
\$2.33 billion - [Mentor Graphics](#)

\$507 million - [Magma Design Automation](#) (now Synopsys)

\$157 million - [Zuken Inc.](#)

Design Stages (FPGA, ASIC)

- **Requirements**
- **Feasibility studies**
- **Hardware specification**
- **Hardware architecture**
- **Coding (Register Transfer Level)**
- **Verification (testbench, etc)**
- **Physical implementation (synthesis, place/layout)**
- **Hardware test and integration**



There are three major FPGA manufacture companies today:

Xilinx: SRAM based FPGA, the configuration will be lost (FPGA becomes blank) when power is off, FPGA need to do re-configuration from EPROM/FLASH first whenever power is on)

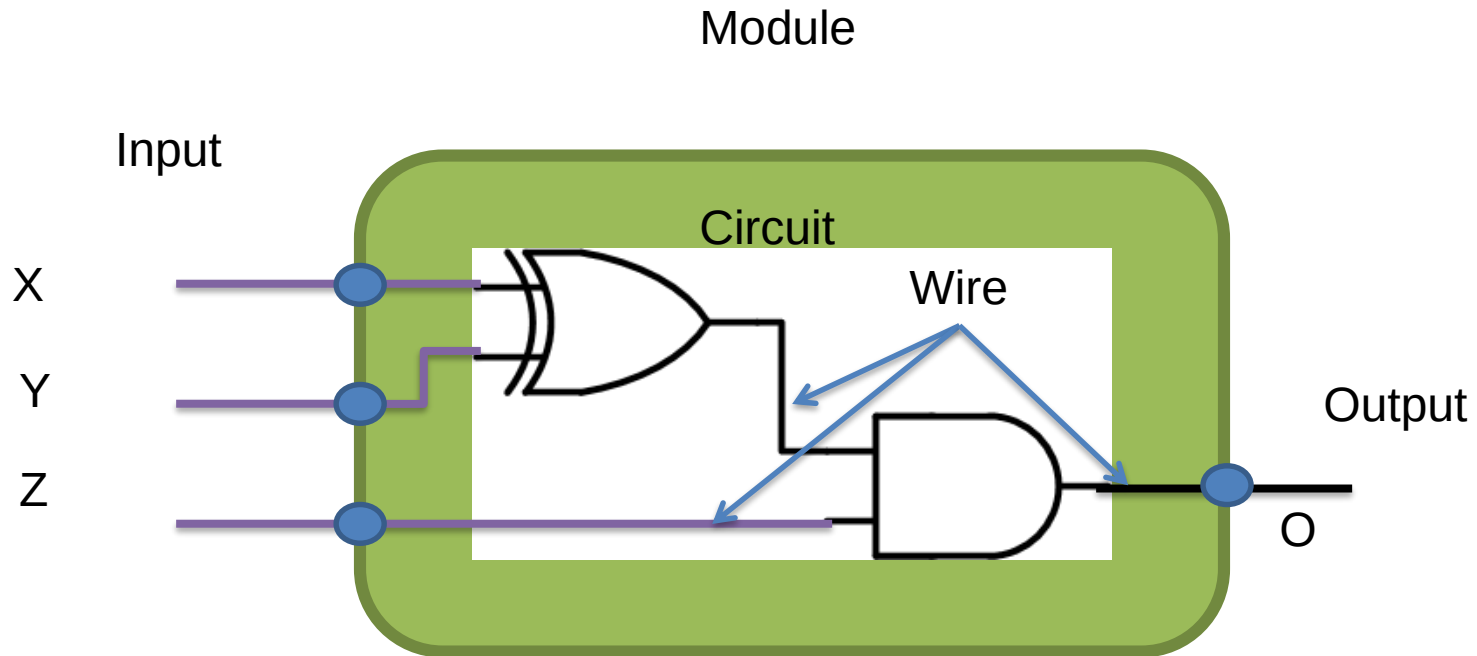
Altera: SRAM based FPGA, but they are called CPLD (Complex Programmable Logic Device)

Actel: Anti-fuse based FPGA, nonvolatile device (configuration is kept after power is off), one time configuration, because their connections between CLBs are physically changed during configuration process, like ASIC. They're also making flash based FPGA that is nonvolatile device but can be re-configured. Both FPGAs don't need re-configuration when power is on.

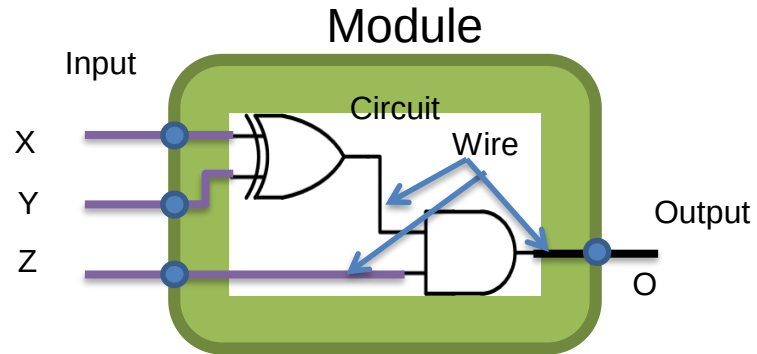
Market share: Xilinx : Altera = 6 : 4

Xilinx president said “Xilinx FPGA has friendly relations with VHDL”

Module declaration



Module declaration



```
module sample (X,Y,Z,O);
```

```
input X,Y,Z;
```

```
output O;
```

```
wire O;
```

```
// Describe the circuit using logic symbols
```

```
assign O = (X^Y)&Z;
```

```
endmodule
```

Module name

Data Types ~ summary

- Data Values:

0,1,x,z

- Wire

- Synthesizes into wires
- Used in structural code

- Reg

- May synthesize into latches, flip-flops or wires
- Used in procedural code

- Integer

32-bit integer used as indexes

- Input, Output, inout

Defines ports of a module (wire by default)

```
module sample (a,b,c,d);
```

```
input a,b;  
output c,d;
```

```
wire [7:0] b;
```

```
reg c,d;
```

```
integer k;
```

Comments

- `// The rest of the line is a comment`
- `/* Multiple line
comment */`
- `/* Nesting /* comments */ do NOT work */`

Verilog Value Set

- *0* represents low logic level or false condition
- *1* represents high logic level or true condition
- *x* represents unknown logic level, don't care
- *z* represents high impedance logic level

Data Values

- Numbers:

Numbers are defined by number of bits

Value of 23:

5'b10111 // Binary

5'd23 // Decimal

5'h17 // Hex

5'o27 // Octal

- Constants:

```
wire [3:0] t,d;
```

```
assign t = 23;
```

```
assign d= 4'b0111;
```

- Parameters:

```
parameter n=4;
```

```
localparam m = 5;
```

```
wire [n-1:0] t, d;
```

Vectors

- Represent buses

```
wire [3:0] busA;  
reg [1:4] busB;  
reg [1:0] busC;
```

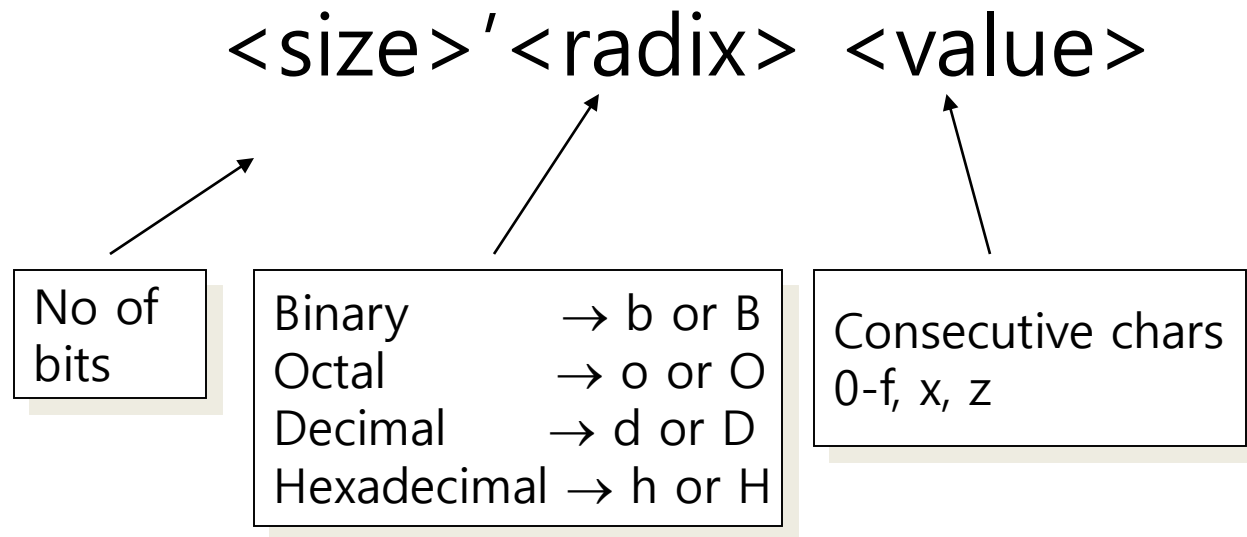
- Left number is MS bit
- Slice management

$$\text{busC} = \text{busA}[2:1]; \quad \Leftrightarrow \quad \begin{cases} \text{busC}[1] = \text{busA}[2]; \\ \text{busC}[0] = \text{busA}[1]; \end{cases}$$

- Vector assignment (***by position!!***)

$$\text{busB} = \text{busA}; \quad \Leftrightarrow \quad \begin{cases} \text{busB}[1] = \text{busA}[3]; \\ \text{busB}[2] = \text{busA}[2]; \\ \text{busB}[3] = \text{busA}[1]; \\ \text{busB}[4] = \text{busA}[0]; \end{cases}$$

Numbers in Verilog (i)



- `8'h ax = 1010xxxx`
- `12'o 3zx7 = 011zzzxxx111`

Operators

- Arithmetic:

*****, **+**, **-**, **/**, **%**

- Relational

<, **<=**, **>**, **>=**, **==**, **!=**

- Bit-wise Operators

- Not: **~**

- XOR: **^**

- And : **&** 5'b11001 & 5'b01101 ==>
5'b01001

- OR: **|**

- XNOR: **~^** or **^~**

- Logical Operators

Returns 1 or 0, treats all nonzero as 1

- **!** : Not

- **&&** : AND 27 && -3 ==> 1

- **||** : OR

```
reg [3:0] a, b, c, d;  
wire[7:0] x,y,z;  
parameter n =4;
```

```
c = a + b;  
d = a *n;
```

```
if(x==y) d = 1; else d =0;
```

```
d = a ~^ b;
```

```
if ((x>=y) && (z)) a=1;  
else a = !x;
```

Operators

- Reduction Operators:

Unary operations returns single-bit values

- **&** : and
- **|** :or
- **~&** : nand
- **~|** : nor
- **^** : xor
- **~^** :xnor

- Shift Operators

Shift Left: <<

Shift right: >>

- Concatenation Operator

{ } (concatenation)

{ **n{item}** } (n fold replication of an item)

- Conditional Operator

Implements if-then-else statement

(cond) ? (result if cond true) : (result if cond false)

```
module sample (a, b, c, d);  
input [2:0] a, b;  
output [2:0] c, d;  
wire z,y;
```

```
assign z = ~| a;  
c = a * b;  
if(a==b) d = 1; else d =0;
```

```
d = a ~^ b;
```

```
if ((a>=b) && (z)) y=1;  
else y = !x;
```

```
assign d << 2; //shift left  
twice
```

```
assign {carry, d} = a + b;  
assign c = {2{carry},2{1'b0}};  
// c = {carry,carry,0,0}
```

```
assign c= (inc==2)? a+1:a-1;
```

Reduction Operators

- $\&$ $\rightarrow$ AND
- $|$ $\rightarrow$ OR
- $\wedge$ $\rightarrow$ XOR
- $\sim\&$ $\rightarrow$ NAND
- $\sim|$ $\rightarrow$ NOR
- $\sim\wedge$ or $\wedge\sim$ $\rightarrow$ XNOR
- One multi-bit operand $\rightarrow$ One single-bit result

```
a = 4'b1001;
```

```
..
```

```
c = |a; // c = 1|0|0|1 = 1
```

Shift Operators

- `>>` → shift right
- `<<` → shift left
- Result is same size as first operand, **always zero filled**

```
a = 4'b1010;
```

```
...
```

```
d = a >> 2;    // d = 0010
```

```
c = a << 1;    // c = 0100
```

Concatenation Operator

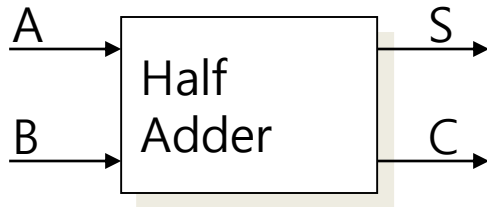
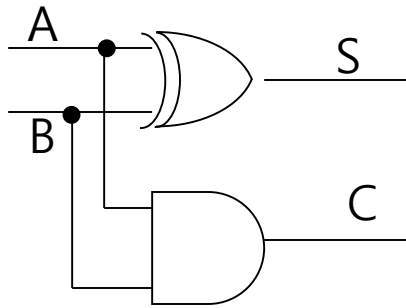
- $\{op1, op2, ..\} \rightarrow$ concatenates $op1, op2, ..$ to single number
- Operands must be sized !!

```
reg a;  
reg [2:0] b, c;  
..  
a = 1'b 1;  
b = 3'b 010;  
c = 3'b 101;  
catx = {a, b, c};           // catx = 1_010_101  
caty = {b, 2'b11, a};       // caty = 010_11_1  
catz = {b, 1};              // WRONG !!
```

- Replication ..

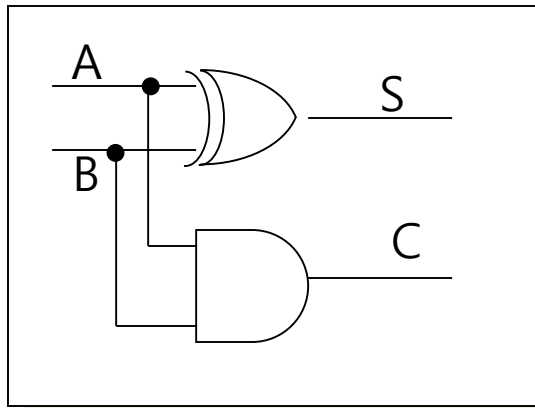
```
catr = {4{a}, b, 2{c}};     // catr = 1111_010_101101
```

Example: Half Adder



```
module half_adder(S, C, A, B);  
  output S, C;  
  input A, B;  
  
  wire S, C, A, B;  
  
  assign #2 S = A ^ B;  
  assign #1 C = A & B;  
  
endmodule
```

Example: Half Adder, 2nd Implementation

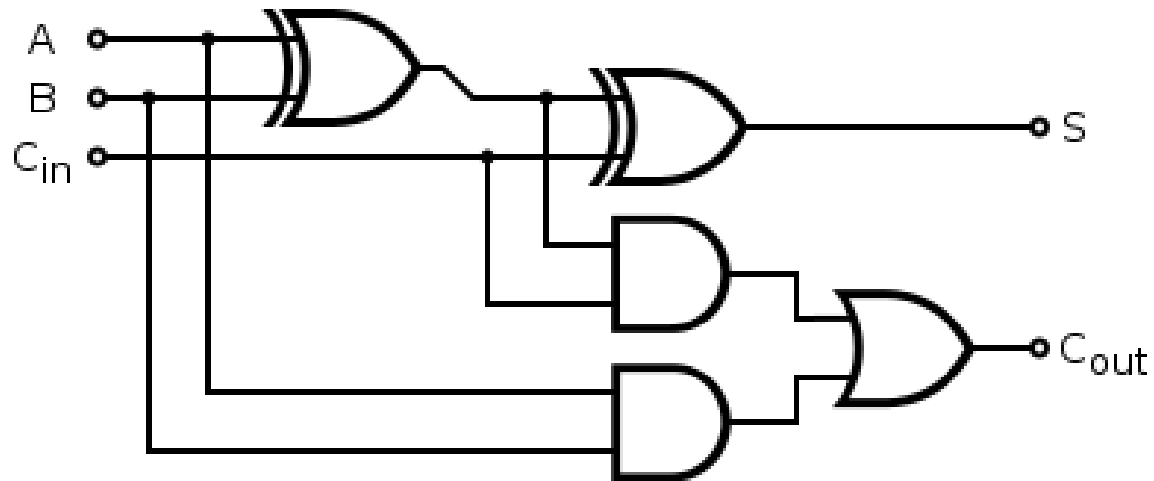


Assuming:

- XOR: 2 t.u. delay
- AND: 1 t.u. delay

```
module half_adder(S, C, A, B);  
  output S, C;  
  input A, B;  
  
  wire S, C, A, B;  
  
  xor #2 (S, A, B);  
  and #1 (C, A, B);  
  
endmodule
```

Example: Full adder



Example: Full Adder

```
module full_adder(a, b, cin, s, cout);  
  input a, b, cin;  
  output s, cout;  
  
  wire s1, s2, s3;  
  
  assign s1 = a ^ b;  
  assign s = s1 ^ cin;  
  assign s2 = s1 & cin;  
  assign s3 = a & b;  
  assign cout = s2 | s3;  
  
endmodule
```

Example: 4-to-1 multiplexor

